ABSTRACT

Learning Algorithms for Automatic Data Structure Design

DEMI GUO, Harvard University, Cambridge, MA, United States

Published: 18 June 2021

SIGMOD/PODS '21: International Conference on Management of Data
June 20 - 25, 2021
Virtual Event, China

Conference Sponsors:
SIGMOD

SIGMOD '21: Proceedings of the 2021 International Conference on Management of Data (June 2021)
https://doi.org/10.1145/3448016.3450570
ISBN: 9781450383431

Learning Algorithms for Automatic Data Structure Design

Demi Guo*
Harvard University

ABSTRACT

We present practical learning-based search algorithms that make it possible to automatically design key-value data structures. Our work allows searching within a space of 10^{100} possible designs for the optimal data structure. The input is a workload specification and the output is an abstract syntax tree which can be absorbed by modern code-generation techniques. Given the vast design space our solution is based on learning and we show that it can find the close to optimal data structure design in a matter of a few seconds.

ACM Reference Format:
Demi Guo. 2021. Learning Algorithms for Automatic Data Structure Design. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21), June 20–25, 2021, Virtual Event, China.* ACM, New York, NY, USA, 3 pages. https://doi.org/10.1145/3448016.3450570

1 INTRODUCTION

The Importance of Data Structures. Data structures are at the core of numerous computer science subfields. With the contemporary trends of 1) data growth, 2) the increasing number of data-driven applications, and 3) more fields moving to a computational paradigm where data collection, storage, and analysis become necessary, data structure design has become more critical now than ever [10, 11, 19, 20].

The Vast Design Space Slows Progress. Applications today mostly rely on a manual process of selecting data structures based on experience or prior expertise. Given that the design space is vast, not only is this process time-consuming but manually selected designs also turn out to be sub-optimal thereby affecting performance [19].

The Data Calculator. Recent research has made it easier to reason about the design space of data structures by introducing the *Data Calculator* [19] that identifies the fundamental design primitives spanning over more than trillions (10^{18}) of possible data structures.

Problem: How to design data structures automatically? The Data Calculator gives the deign space and models to evaluate the designs but because the space is massive, we also need search algorithms to allow for efficient navigation. For example, one may use a brute-force strategy. However, such an approach will not only be (i) significantly time-consuming but more importantly, (ii) it is theoretically impossible to navigate through all possible enumerations of design primitives that are of continuous nature such as storage or memory footprint and thus, (iii) ensuring close to optimal designs is very hard with brute-force.

*We thank Stratos Idreos and Subarna Chatterjee for advising this project.

SIGMOD '21, June 20–25, 2021, Virtual Event, China

ACM ISBN 978-1-4503-8343-1/21/06.
https://doi.org/10.1145/3448016.3450570

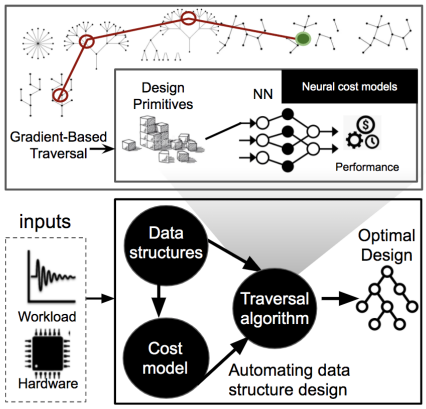

Figure 1: Designing efficient search algorithms is crucial for automating data structure selection.

Solution: Learning-Based Algorithms for Efficient Search. We propose a search algorithm with neural network-based learning followed by efficient traversal through the design space with the help of gradient-descent optimization. We show that (i) the search algorithm can accurately learn the performance of arbitrary data structure designs and (ii) can propose better designs as opposed to a brute-force search approach for diverse workloads.

2 DATA STRUCTURE DESIGN SEARCH

Problem Formulation. We formulate the search problem as follows: We accept a workload w and hardware specifications h as input. The output is the optimal data structure design that renders the maximum performance, i.e., high throughput. See Figure 1 for an illustration. We define θ to be a continuous vector representing a data structure in the possible design space D, and $Q(w, h, \theta)$ to be the performance offered by this data structure for the given input setting. Then, finding the optimal data structure can be written as:

$$\tilde{\theta} = \arg\max_{\theta} Q(w, h, \theta) \quad (1)$$

For the composition of a data structure design, we rely on the primitives and cost models of [8] that describes a design in terms of the following knobs: the growth factor between levels of the data structure (T), the greediness of merging data (K, Z) within hot and cold levels, respectively, and the memory allocated to buffer (M_B), indexes (M_{FP}), and filters (M_F). These knobs allow a structured description of any data structure spanning across the consolidated space of B-trees [2], LSM-trees [17], BW-trees [12], LSH-tables [7] as well as their hybrids [1, 3–5, 11, 13, 20].

Intuition for the Algorithm Design. Directly finding the optimal $\tilde{\theta}$ in equation (1) is intractable, because there can be infinite choices of $\theta \in D$ given θ can be continuous (e.g. memory allocation ratio). Brute force enumerates over a discretized finite subspace, yet it can still be expensive and fundamentally dismisses certain design choices. The difficulty of finding the optimal $\tilde{\theta}$ efficiently lies in the fact that the performance $Q(w, h, \theta)$ is a black-box function. One

Algorithm 1: Data Structure Design Search.

Input : workload w, hardware specification h
Output: data structure design $\tilde{\theta}$
Sample w, h, θ to create a dataset D_S;
repeat
  Sample a batch of data D'_S from D_S ;
  Update ϕ by performing a gradient step to minimize loss $\frac{1}{|D'_S|}\sum_{w,h,\theta \in D'_S} \|f(w,h,\theta) - Q(w,h,\theta)\|^2$;
until *convergence*;
Initialize $\theta^{(0)}$;
for $t \leftarrow 0$ **to** $U-1$ **do**
  $\theta^{(t+1)} = \theta^{(t)} + \alpha \frac{\partial}{\partial \theta^{(t)}} f(w,h,\theta)$
end
return $\theta^{(U)}$;

common approach for black-box objective optimization is Bayesian Optimization, which however can still be problematic when the search space D is high-dimensional [18].

Search Algorithm. To address these challenges, we propose a novel algorithm shown in Algorithm 1. It first learns a neural cost model $f_\phi(w,h,\theta)$ parameterized by ϕ to estimate performance $Q(w,h,\theta)$. We create a dataset $D_S \subseteq D$ by sampling w,h,θ and compute the corresponding $Q(w,h,\theta)$. Since our goal is to use f to approximate Q, we capture the accuracy of the model by measuring the the mean-squared loss. Given its nature as a regression problem, we use a small multilayer perceptron (a class of feedforward artificial neural network) [15] to parametrize the neural model f.

The second step of the algorithm is to efficiently navigate the search space for optimal design by performing gradient descent on $f_\phi(w,h,\theta)$. We treat parameters of f to be fixed, and employ stochastic gradient descent over the now learnable parameters θ to minimize the neural cost model, i.e. $\min_\theta f(s,\theta)$. We initialize θ with an arbitrary design choice $\theta^{(0)}$, and at each step t update θ with the formula shown in the for loop in Algorithm 1 where α is a hyper-parameter representing step size. In total, we repeat this update step for U times, where U is a hyper-parameter. The high-level overview of the algorithm is illustrated in Figure 1.

3 EXPERIMENTAL EVALUATION

We now demonstrate that the neural cost model can automatically find efficient data structure designs.

Experimental Setup. We generate three synthetic datasets, representing workloads with diverse portions of reads and writes. For each dataset, we sample hundreds of different memory and data settings instances, and sample thousands of value assignments for design knobs $\theta = (T, K, Z, M_B)$. The entire dataset consists of around 50,000 example designs, which is sufficiently large for training our MLP regressor following previous works [6, 16]. We then split the datasets for training, validation, and testing. For training, we use a multilayer perceptron and an adam optimizer [9]. We tune a small set of hyper-parameters (i.e. batch size, learning rate, number of layers, hidden size) on validation dataset. All experiments are conducted on a P100 NVIDIA GPU on Azure.

The neural cost model is significantly accurate. We first evaluate the consistency of design rankings provided by the neural model against the analytical cost models of [8]. We use a ranking metric popular in retrieval problems [14], *AR@K*, that computes the average recall at top K designs:

$$\text{AR@K} = \frac{|\text{analytical top K designs} \cap \text{neural top K designs}|}{|K|} \quad (2)$$

Intuitively, a higher AR@K implies the learned neural cost model can better preserve the rankings of original analytical cost function. The experiments are performed on a held-out validation dataset and the results are shown in Table 1. Even though there are 4,000 different design options to rank, which implies the chance baseline for AE@1 is 0.025%, our neural cost model is able to rank designs similarly as the analytical cost function.

Workload	Top k%					BC (%)
	1	5	10	50	100	
write-intensive (20% reads, 80% writes)	18.3	17.3	19.6	31.7	28.9	30.7
mixed (50% reads, 50% writes)	35.6	34.1	30.5	37.1	32.0	31.5
read-intensive (80% reads, 20% writes)	47.8	20.3	22.6	34.4	38.7	32.5

Table 1: The neural network based search algorithm can accurately predict performance of different designs and can suggest better design compared to brute-force search.

We can find better solutions than brute force. To verify the effectiveness of the algorithm, we first initialize $\theta^{(0)}$ with a reasonable valid design (which is likely suboptimal), and evaluate whether the final design $\theta^{(U)}$ proposed by our algorithm is more optimal than the best solution found by brute force. We enumerate through different hardware/workload settings, and quantify the quality of the solution with *BC*: *percentage of settings when the algorithm provides better solution than brute force.*

Results are shown in Table 1. For all three different types of workloads, there is around 30% chance that the algorithm can improve the data structure design. This is because brute force cannot find the optimal design as it is not able to (practically) iterate over all possible enumerations of continuous knobs (e.g. memory allocation across buffer, filters and indexes), while our algorithm is not bounded by a discretized search space. In cases of improvements, we found that our algorithm picked designs that had up to 20% performance improvement on average.

Response Time. At the same time, we observe that on an average across the three different workloads, brute force takes about 349 seconds while our algorithm takes 25 seconds making the execution about 14X faster.

Summary. We showed early signs that a search method for automatic data structure design based on neural networks can be beneficial. The proposed model sets off by imitating the analytical cost model and as more real-world data from diverse applications is fed to it, the model gets through the learning curve, fine-tuning its accuracy to the point where it significantly outperforms the analytical models. Next steps include application of our algorithm in larger design space (beyond [8]), the integration of cloud cost in the metrics and concurrency in the primitives.

REFERENCES

[1] Mayank Bawa, Tyson Condie, and Prasanna Ganesan. 2005. LSH forest: self-tuning indexes for similarity search.. In *WWW*, Allan Ellis and Tatsuya Hagino (Eds.). ACM, 651–660. http://dblp.uni-trier.de/db/conf/www/www2005.html#BawaCG05

[2] R. Bayer and E. McCreight. 1970. Organization and Maintenance of Large Ordered Indices. In *Proceedings of the 1970 ACM SIGFIDET (Now SIGMOD) Workshop on Data Description, Access and Control* (Houston, Texas) *(SIGFIDET '70)*. Association for Computing Machinery, New York, NY, USA, 107–141. https://doi.org/10.1145/1734663.1734671

[3] Michael A. Bender, Yonatan R. Fogel, Martin Farach-Colton, Bradley C. Kuszmaul, and et al. 2007. Cache-Oblivious Streaming B-trees.

[4] Douglas Comer. 1979. The Ubiquitous B-Tree. *ACM Comput. Surv.* 11, 2 (1979), 121–137. http://dblp.uni-trier.de/db/journals/csur/csur11.html#Comer79

[5] Niv Dayan, Manos Athanassoulis, and Stratos Idreos. 2017. Monkey: Optimal Navigable Key-Value Store. In *Proceedings of the 2017 ACM International Conference on Management of Data, SIGMOD Conference 2017, Chicago, IL, USA, May 14-19, 2017*, Semih Salihoglu, Wenchao Zhou, Rada Chirkova, Jun Yang, and Dan Suciu (Eds.). ACM, 79–94. https://doi.org/10.1145/3035918.3064054

[6] Dheeru Dua and Casey Graff. 2017. UCI Machine Learning Repository. http://archive.ics.uci.edu/ml

[7] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. 1999. Similarity Search in High Dimensions via Hashing. In *Proceedings of the 25th International Conference on Very Large Data Bases (VLDB '99)*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 518–529.

[8] S. Idreos, N. Dayan, W. Qin, M. Akmanalp, S. Hilgard, A. Ross, J. Lennon, V. Jain, H. Gupta, D. Li, and Z. Zhu. 2019. Design Continuums and the Path Toward Self-Designing Key-Value Stores that Know and Learn. In *Biennial Conference on Innovative Data Systems Research (CIDR)*.

[9] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, Yoshua Bengio and Yann LeCun (Eds.). http://arxiv.org/abs/1412.6980

[10] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. 2020. Reformer: The Efficient Transformer. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net. https://openreview.net/forum?id=rkgNKkHtvB

[11] Tim Kraska, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. 2017. The Case for Learned Index Structures. *CoRR* abs/1712.01208 (2017). http://dblp.uni-trier.de/db/journals/corr/corr1712.html#abs-1712-01208

[12] Justin J. Levandoski, David B. Lomet, and Sudipta Sengupta. 2013. The Bw-Tree: A B-tree for new hardware platforms.. In *ICDE*, Christian S. Jensen, Christopher M. Jermaine, and Xiaofang Zhou (Eds.). IEEE Computer Society, 302–313. http://dblp.uni-trier.de/db/conf/icde/icde2013.html#LevandoskiLS13a

[13] Y. Li, B. He, Jun Yang, Q. Luo, and K. Yi. 2010. Tree indexing on solid state drives. *Proceedings of the VLDB Endowment* 3 (2010), 1195 – 1206.

[14] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. 2008. *Introduction to Information Retrieval*. Cambridge University Press. http://nlp.stanford.edu/IR-book/

[15] Popescu Marius, Valentina Balas, Liliana Perescu-Popescu, and Nikos Mastorakis. 2009. Multilayer perceptron and neural networks. *WSEAS Transactions on Circuits and Systems* 8 (07 2009).

[16] Sebastian W. Ober and Carl Edward Rasmussen. 2019. Benchmarking the Neural Linear Model for Regression. *CoRR* abs/1912.08416 (2019). arXiv:1912.08416 http://arxiv.org/abs/1912.08416

[17] Patrick E. O'Neil, Edward Cheng, Dieter Gawlick, and Elizabeth J. O'Neil. 1996. The Log-Structured Merge-Tree (LSM-Tree). *Acta Inf.* 33, 4 (1996), 351–385. http://dblp.uni-trier.de/db/journals/acta/acta33.html#ONeilCGO96

[18] B. Shahriari, Kevin Swersky, Ziyu Wang, R. Adams, and N. D. Freitas. 2016. Taking the Human Out of the Loop: A Review of Bayesian Optimization. *Proc. IEEE* 104 (2016), 148–175.

[19] S.Idreos, K.Zoumpatianos, B.Hentschel, M.S. Kester, and D.Guo. 2018. The Data Calculator: Data Structure Design and Cost Synthesis From First Principles, and Learned Cost Models. In *ACM SIGMOD International Conference on Management of Data*.

[20] Zhenjie Zhang, Marios Hadjieleftheriou, Beng Chin Ooi, and Divesh Srivastava. 2010. Bed-tree: an all-purpose index structure for string similarity search based on edit distance. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2010, Indianapolis, Indiana, USA, June 6-10, 2010*, Ahmed K. Elmagarmid and Divyakant Agrawal (Eds.). ACM, 915–926. https://doi.org/10.1145/1807167.1807266